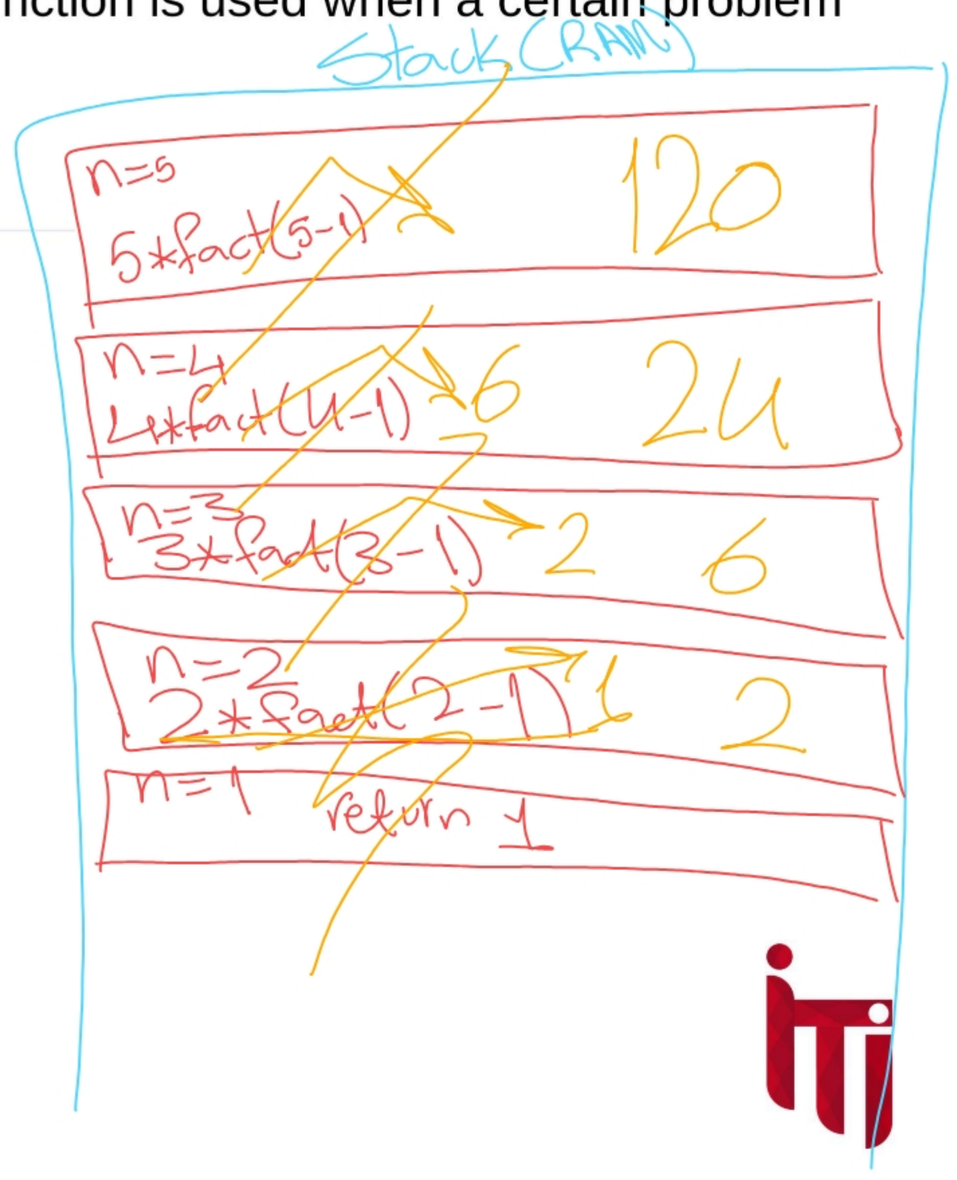
Recursive Function in C

A recursive function in C is a function that calls itself. A recursive function is used when a certain problem is defined in terms of itself.

```c
#include <stdio.h>

// Recursive function for factorial
int factorial(int n) {
    if (n == 0 || n == 1)  // Base case
        return 1;
    else
        return n * factorial(n - 1);  // Recursive case
}

int main() {
    int num = 5;
    printf("Factorial of %d = %d\n", num, factorial(num));
    return 0;
}
```



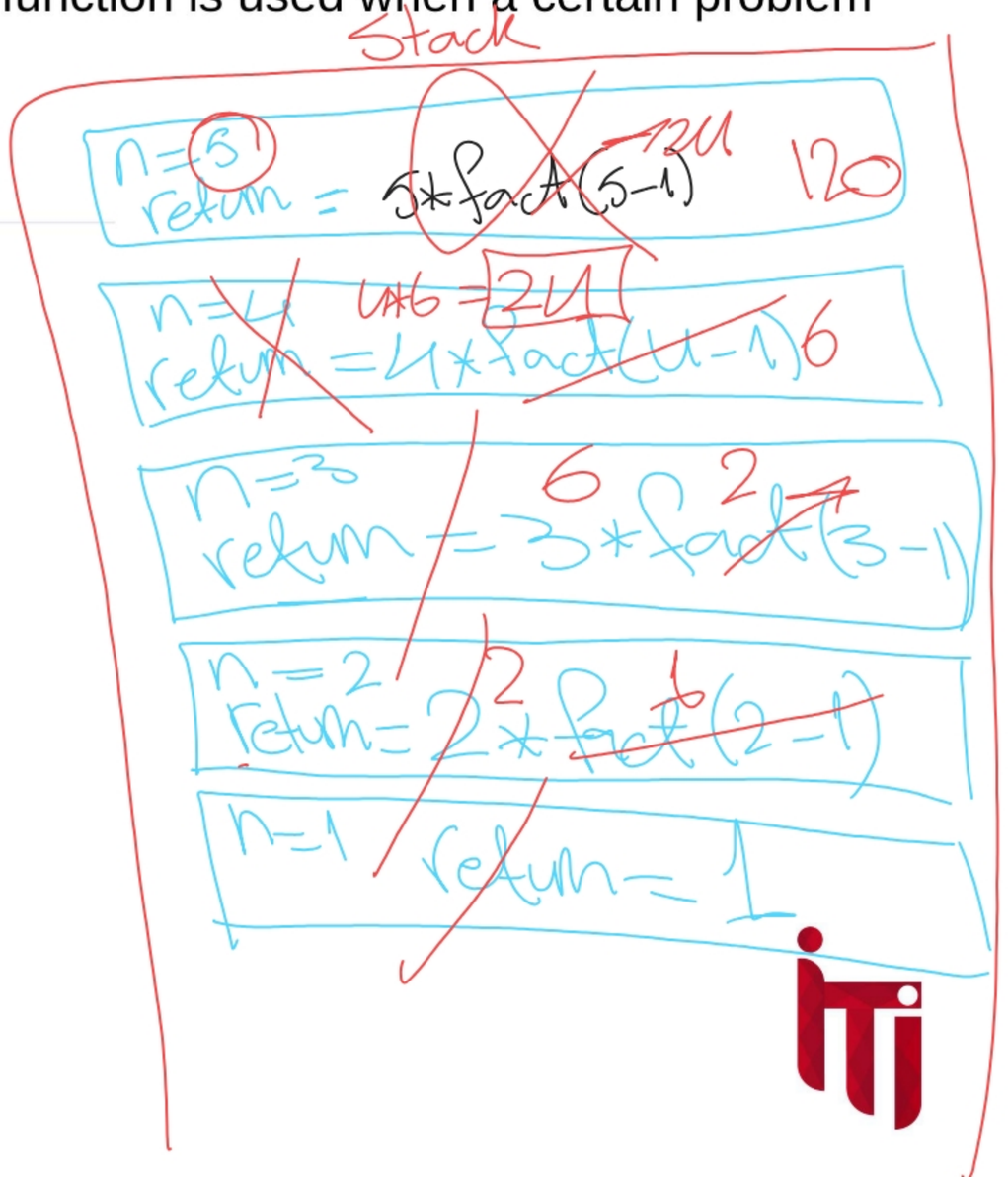
Recursive Function in C

A recursive function in C is a function that calls itself. A recursive function is used when a certain problem is defined in terms of itself.

```c
#include <stdio.h>

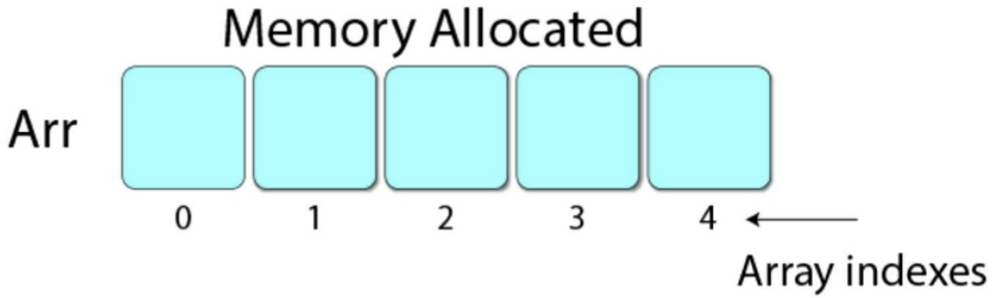
// Recursive function for factorial
int factorial(int n) {
    if (n == 0 || n == 1)  // Base case
        return 1;
    else
        return n * factorial(n - 1);  // Recursive case
}

int main() {
    int num = 5;
    printf("Factorial of %d = %d\n", num, factorial(num));
    return 0;
}
```



Array Declaration

Arr [5]; Size of Array no = 5



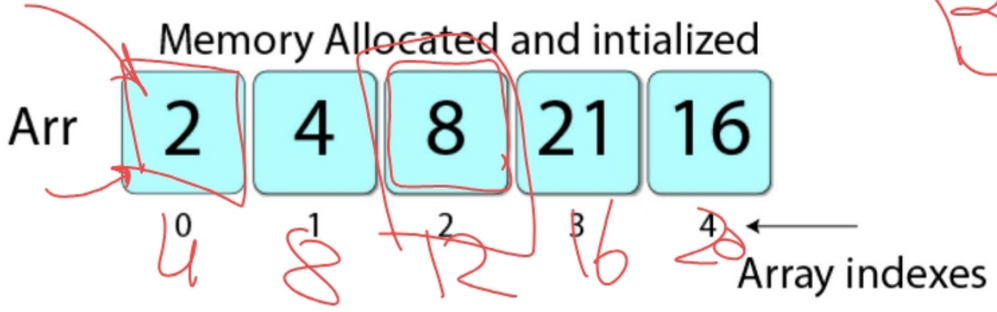
Array intialization

Base

Address

Arr [5] = { 2, 4, 8, 12, 16 };

Base + 2 * size of int



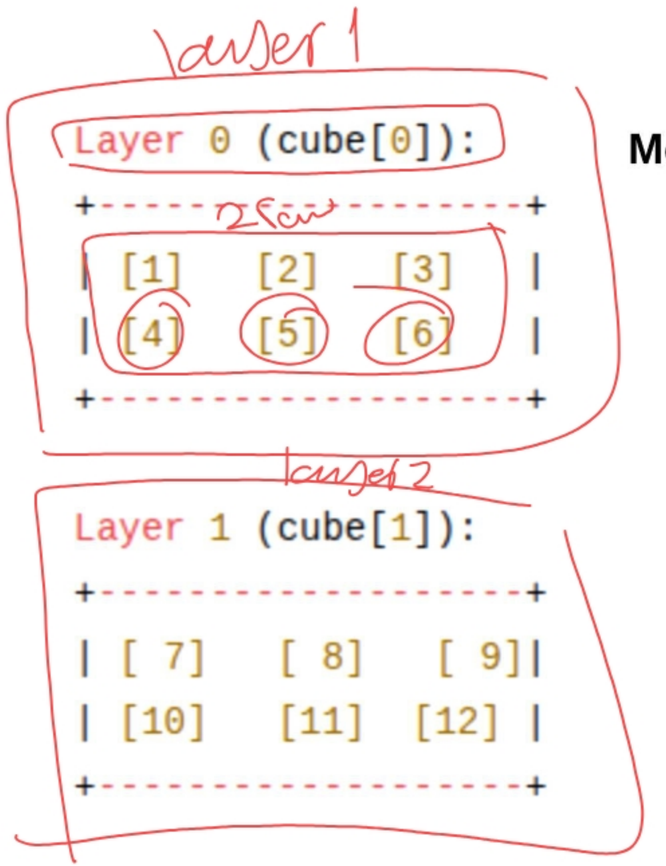
Multidimensional Arrays

A **multi-dimensional array** in C is simply an **array of arrays**. Useful for **matrices, tables, or grids**

```
type name[size1][size2]...[sizeN];
```

```
int matrix[2][3] = {
    {1, 2, 3},
    {4, 5, 6}
};
```

```
int cube[2][2][3] = {
    {
        {1, 2, 3},
        {4, 5, 6}
    },
    {
        {7, 8, 9},
        {10, 11, 12}
    }
};
```



Memory Layout:

- 2 pages
- Each page has 2 rows
- Each row has 3 columns

```
int arr[3][4] = {
    {1, 2,  3,  4},
    {5, 6,  7,  8},
    {9, 10, 11, 12}
};
```

	Row 0				Row 1				Row 2			
Value	1	2	3	4	5	6	7	8	9	10	11	12
Address	1000	1004	1008	1012	1016	1020	1024	1028	1032	1036	1040	1044

12 * size



```
int arr[3][5] = {1,2,3,4,5, 10,20,30,40,50, 5,10,15,20,25};
```

```
Row 0 → [ 1   2   3   4   5 ]
Row 1 → [10  20  30  40  50 ]
Row 2 → [ 5  10  15  20  25 ]
```

```
arr[0][0] = 1

arr[0][4] = 5

arr[1][2] = 30

arr[2][4] = 25
```

```
int arr[][5] = {1,2,3,4,5, 10,20,30,40,50, 5,10,15,20,25};

int arr[][5] = {
    {1,  2,  3,  4,  5},
    {10, 20, 30, 40, 50},
    {5, 10, 15, 20, 25}
};
```

Pass array to function

- Arrays **decay** (automatically convert) into a pointer to their first element when passed to a function.
- This means what actually gets passed is the **address of the first element**, not a full copy.

```c
#include <stdio.h>

int main(){
    int arr[] = {10, 34, 21, 78, 5};
    float avg = average(arr);
    printf("average: %f", avg);
}

float average(int arr[5]){
    int sum=0;
    int i;
    for (i=0; i<5; i++){
        printf("arr[%d]: %d\n", i, arr[i]);
        sum+=arr[i];
    }
    return (float)sum/5;
}
```

```c
#include <stdio.h>

int main(){
    int arr[] = {10, 34, 21, 78, 5};
    int length = sizeof(arr)/sizeof(int);
    float avg = average(arr, length);
    printf("average: %f", avg);
}

float average(int arr[], int length){
    int sum=0;
    int i;
    for (i=0; i<length; i++){
        printf("arr[%d]: %d\n", i, arr[i]);
        sum+=arr[i];
    }
    return (float)sum/length;
}
```



Programiz

C Online Compiler

Programiz PRO ›

main.c

Run ▶

```c
// Online C compiler (editor)
// Write and run C online using this editor.

#include <stdio.h>
int avg(int arr[]){
    int len=sizeof(arr)/sizeof(int);
    printf("len of arr: %i",len);
}
int main() {

    int arr[5]={1,2,3,4,5};
    avg(arr);
    printf("Try clicking the Run button.");
    return 0;
}
```

Output

```
/tmp/x7aJgyZquT-xyQSXJp8/main.c: In function 'avg':
/tmp/x7aJgyZquT-xyQSXJp8/main.c:6:19: warning: 'sizeof' on array function parameter 'arr' will return size of 'int *' [-Wsizeof-array-argument]
    6 |     int len=sizeof(arr)/sizeof(int);
      |                   ^
/tmp/x7aJgyZquT-xyQSXJp8/main.c:5:13: note: declared here
    5 | int avg(int arr[]){
      |         ~~~~^~~~~
len of arr: 2Try clicking the Run button.
```

5. Searching in Strings

- **`strchr(s, c)`** → finds first occurrence of character c.
- **`strrchr(s, c)`** → finds last occurrence of character c.
- **`strstr(s1, s2)`** → finds first occurrence of substring s2 inside s1.

6. Tokenizing Strings

- **`strtok(s, delim)`** → splits a string into tokens using delimiters.

```c
char str[] = "red,green,blue";
char *token = strtok(str, ",");
while(token != NULL) {
    printf("%s\n", token);
    token = strtok(NULL, ",");
}
```



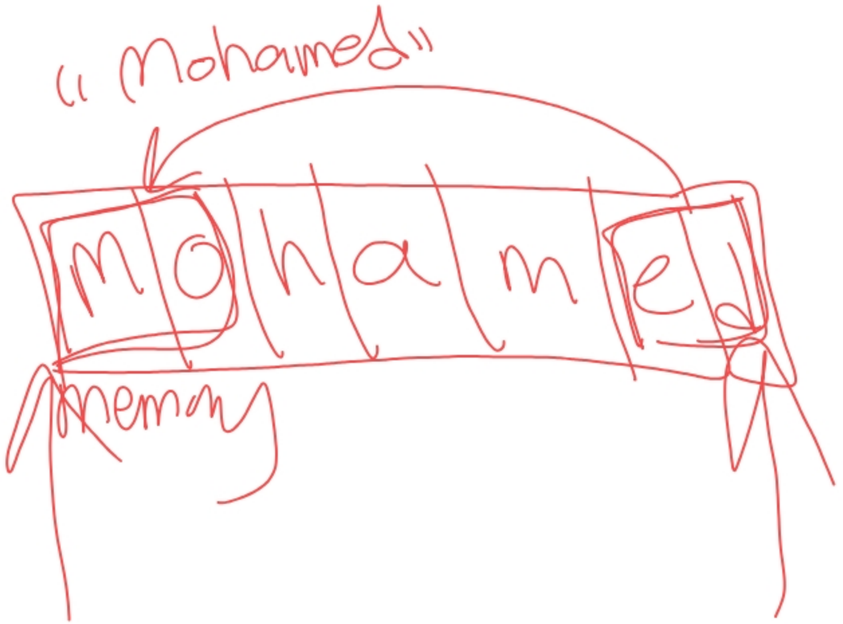
7. Memory Operations

- **`memcpy(dest, src, n)`** → copies n bytes from `src` to `dest`.
- **`memmove(dest, src, n)`** → safer copy (handles overlapping memory).
- **`memcmp(s1, s2, n)`** → compares first n bytes.
- **`memset(s, c, n)`** → fills memory with character c.

```c
#include <stdio.h>
#include <string.h>

int main() {
    char src[] = "Hello";
    char dest[10];

    memcpy(dest, src, 6);  // copy 6 bytes (including '\0')
    printf("dest: %s\n", dest);  // Output: Hello
    return 0;
}
```



```c
int main() {
    int num = 10;       // normal variable
    int *p = &num;      // pointer storing address of num

    printf("Value of num: %d\n", num);      // 10
    printf("Address of num: %p\n", &num);  // some address
    printf("Value stored in pointer p: %p\n", p);   // same address
    printf("Value pointed to by p: %d\n", *p);      // 10

    return 0;
}
```

1. `int num = 10;` → `num` stores the value **10**.
2. `int *p = #` → `p` is a pointer that stores the **address of num**.
3. `*p` → this is called **dereferencing**, which means accessing the value stored at the address.

2. Void Pointer (Generic Pointer)

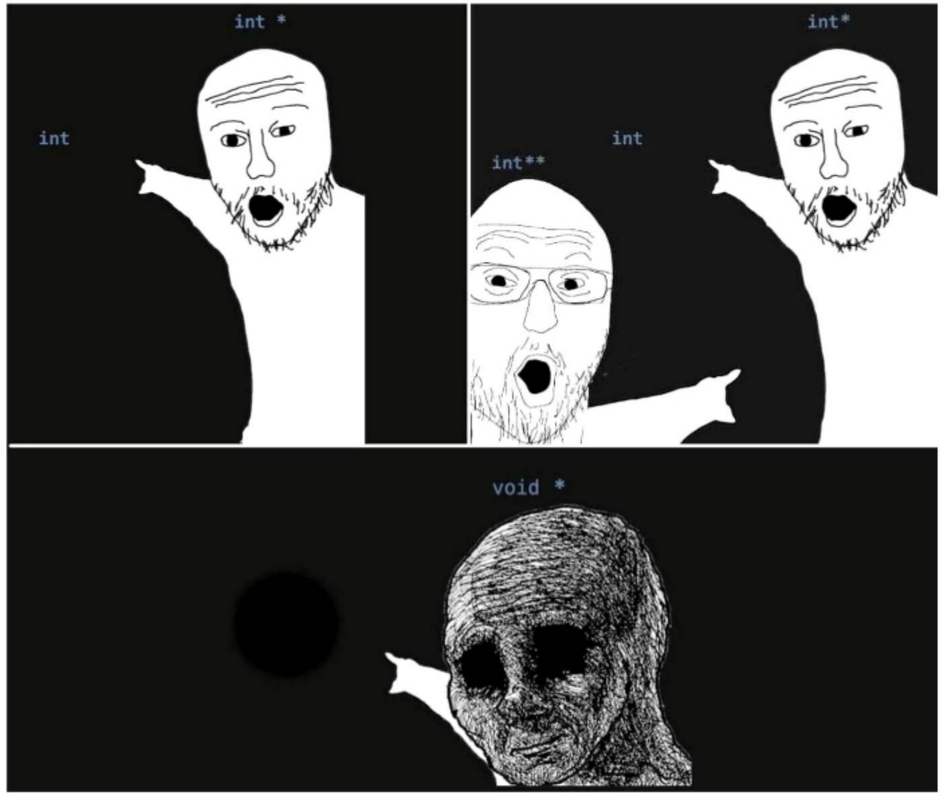
- Special pointer type `void *` that can hold the address of any data type.
- Must be typecast before dereferencing.

```c
#include <stdio.h>
int main() {
    int x = 10;
    float y = 5.5;
    void *ptr;

    ptr = &x;
    printf("Value of x = %d\n", *(int *)ptr);

    ptr = &y;
    printf("Value of y = %.1f\n", *(float *)ptr);

    return 0;
}
```



```
int *Ptr = NULL;

int func()
{
    int x = 10;
    Ptr = &x;
    Printf("%d", *Ptr);
}
Ptr = NULL;
Printf("%d", *Ptr);
```

Ptr → Dangling

Ptr → UN defined

4. Wild Pointer

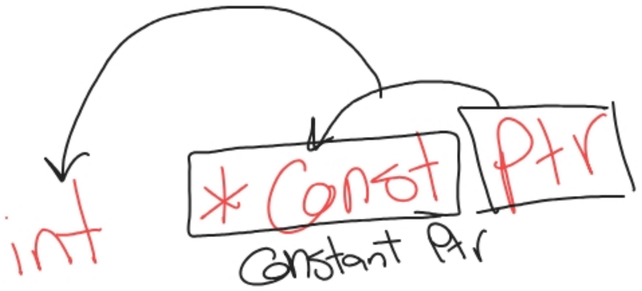
- Uninitialized pointer (points to a random location).

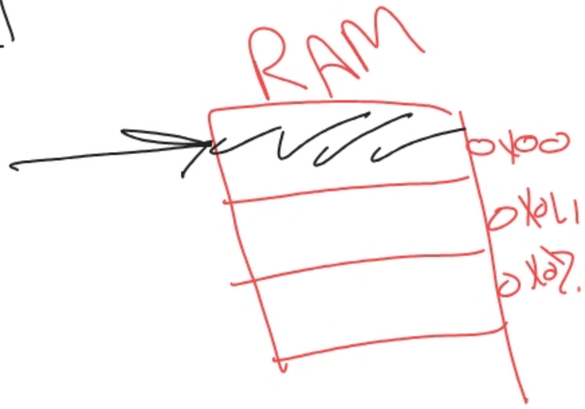
```c
#include <stdio.h>
int main() {
    int *p;    // wild pointer (not initialized)
    // *p = 10; // ❌ dangerous, undefined behavior
    p = NULL;  // ✅ good practice
    return 0;
}
```

5. Constant Pointer

- Pointer itself **cannot change** the address it holds.

```c
#include <stdio.h>
int main() {
    int a = 10, b = 20;
    int * const p = &a;  // constant pointer
    *p = 15;  // allowed (value can change)
    // p = &b; // ❌ error (address can't change)
    return 0;
}
```

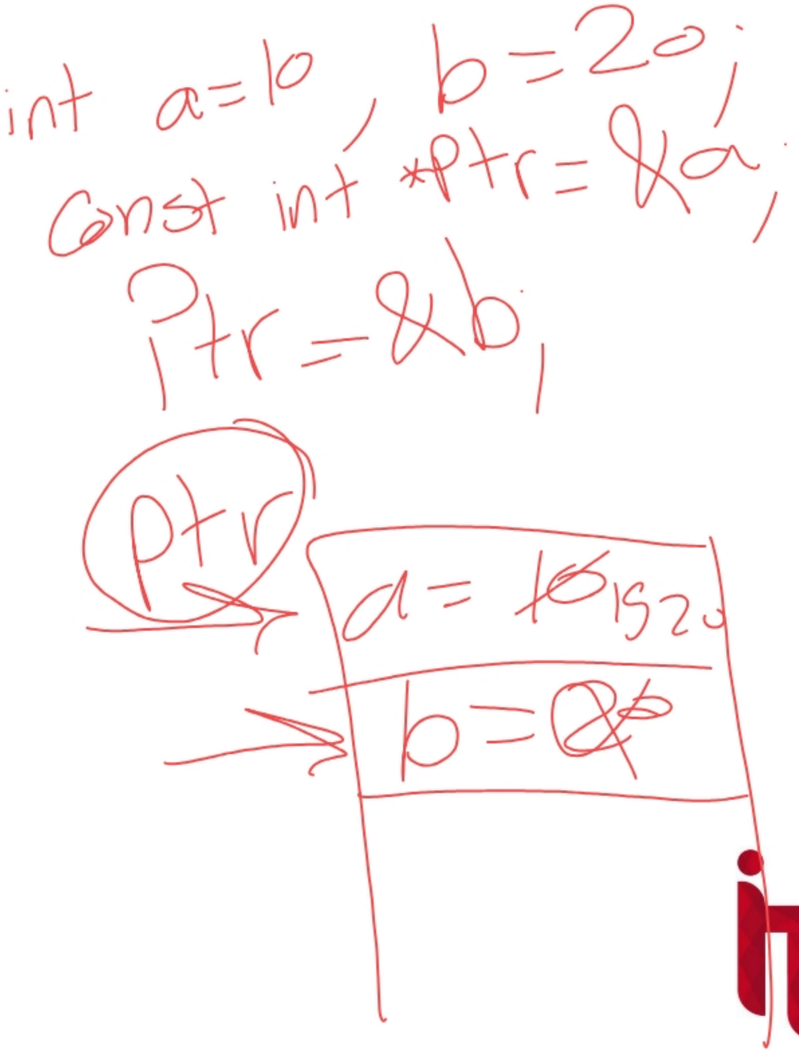




6. Pointer to Constant

- Pointer can move to other variables, but **cannot modify the value** it points to.

```c
#include <stdio.h>
int main() {
    int a = 10, b = 20;
    const int *p = &a;
    // *p = 15; // ❌ error (can't modify value)
    p = &b;     // ✅ allowed (can point elsewhere)
    return 0;
}
```



7. Function Pointer

- Holds the address of a function, used for callbacks.

```c
#include <stdio.h>
void hello() {
    printf("Hello, Function Pointer!\n");
}
int main() {
    void (*fptr)() = hello;
    fptr();  // call function using pointer
    return 0;
}
```

6. Pointer to Constant

- Pointer can move to other variables, but **cannot modify the value** it points to.

```c
#include <stdio.h>
int main() {
    int a = 10, b = 20;
    const int *p = &a;
    // *p = 15; // ❌ error (can't modify value)
    p = &b;     // ✅ allowed (can point elsewhere)
    return 0;
}
```



7. Function Pointer

- Holds the address of a function, used for callbacks.

```c
#include <stdio.h>
void hello() {
    printf("Hello, Function Pointer!\n");
}
int main() {
    void (*fptr)() = hello;
    fptr();  // call function using pointer
    return 0;
}
```

6. Pointer to Constant

- Pointer can move to other variables, but **cannot modify the value** it points to.

```c
#include <stdio.h>
int main() {
    int a = 10, b = 20;
    const int *p = &a;
    // *p = 15; // ❌ error (can't modify value)
    p = &b;     // ✅ allowed (can point elsewhere)
    return 0;
}
```



7. Function Pointer

- Holds the address of a function, used for callbacks.

```c
#include <stdio.h>
void hello() {
    printf("Hello, Function Pointer!\n");
}
int main() {
    void (*fptr)() = hello;
    fptr();  // call function using pointer
    return 0;
}
```

Pointer Arithmetic

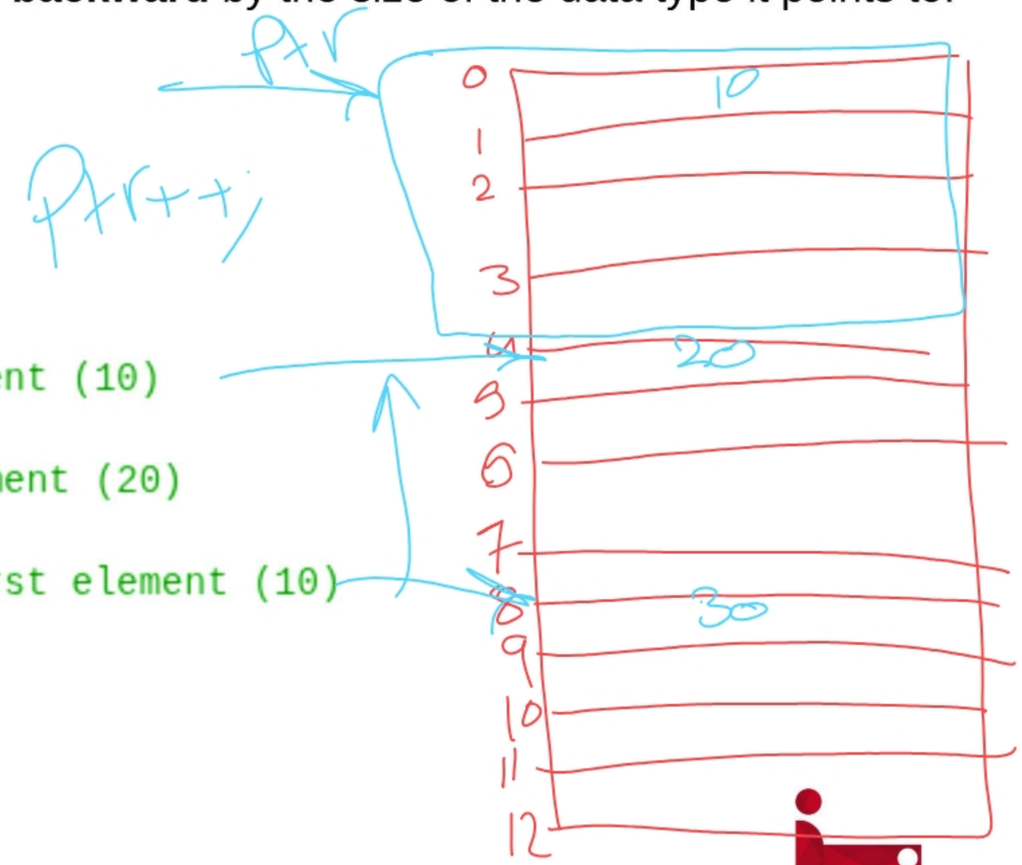
1. Increment and Decrement of a Pointer

When you increment (p++) or decrement (p--) a pointer, it moves **forward or backward** by the size of the data type it points to.

```c
#include <stdio.h>
int main() {
    int arr[3] = {10, 20, 30};
    int *p = arr;

    printf("Address: %p, Value: %d\n", p, *p);   // first element (10)
    p++;  // move to next element
    printf("Address: %p, Value: %d\n", p, *p);   // second element (20)
    p--;  // move back
    printf("Address: %p, Value: %d\n", p, *p);   // back to first element (10)

    return 0;
}
```



- The pointer moves **+4 bytes** or **–4 bytes** (on 32-bit system) because `int` = 4 bytes.

2. Addition and Subtraction of Integer to Pointer

You can add or subtract an integer n from a pointer. This moves the pointer **n elements forward or backward**.

```c
#include <stdio.h>
int main() {
    int arr[5] = {10, 20, 30, 40, 50};
    int *p = arr;

    printf("Value: %d\n", *(p + 2));  // jumps 2 elements → 30
    printf("Value: %d\n", *(p + 4));  // jumps 4 elements → 50
    printf("Value: %d\n", *(p + 0));  // same as *p → 10

    return 0;
}
```

- Here, `(p + 2)` means **move 2 * sizeof(int) bytes ahead**.

3. Subtraction of Pointers

You can subtract one pointer from another **if both point to elements of the same array**. The result is the **number of elements between them**.




```c
#include <stdio.h>
int main() {
    int arr[5] = {10, 20, 30, 40, 50};
    int *p = &arr[4];   // points to last element (50)
    int *q = &arr[1];   // points to second element (20)

    int diff = p - q;   // difference in terms of elements
    printf("Difference = %d\n", diff);

    return 0;
}
```

- Output: `Difference = 3`
  (Because p is 3 elements ahead of q in the array.)

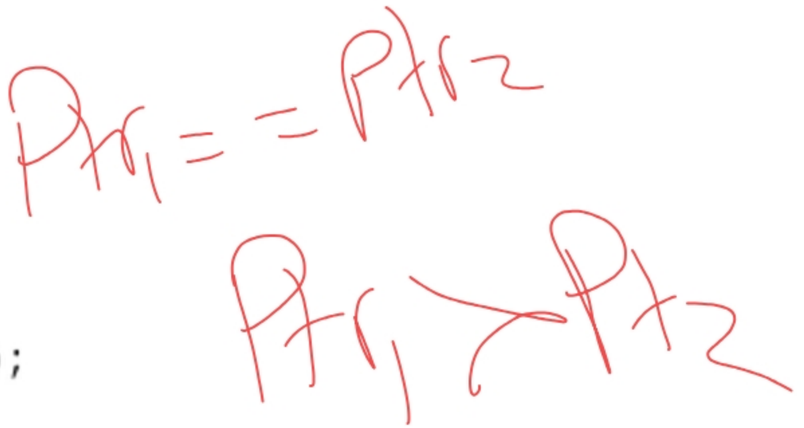
4. Comparison of Pointers

You can compare pointers with relational operators (==, !=, <, >) **only if they point into the same array**.
This checks **which element comes first** in memory.

```c
#include <stdio.h>
int main() {
    int arr[5] = {10, 20, 30, 40, 50};
    int *p = &arr[1];   // points to 20
    int *q = &arr[3];   // points to 40

    if (p < q)
        printf("p points to an earlier element than q\n");
    else
        printf("p points to a later element than q\n");

    return 0;
}
```



Most common uses of pointers

1. To Access Array Elements

- Array name is itself a pointer to the first element.

```c
#include <stdio.h>
int main() {
    int arr[5] = {10, 20, 30, 40, 50};
    int *p = arr;  // pointer to array

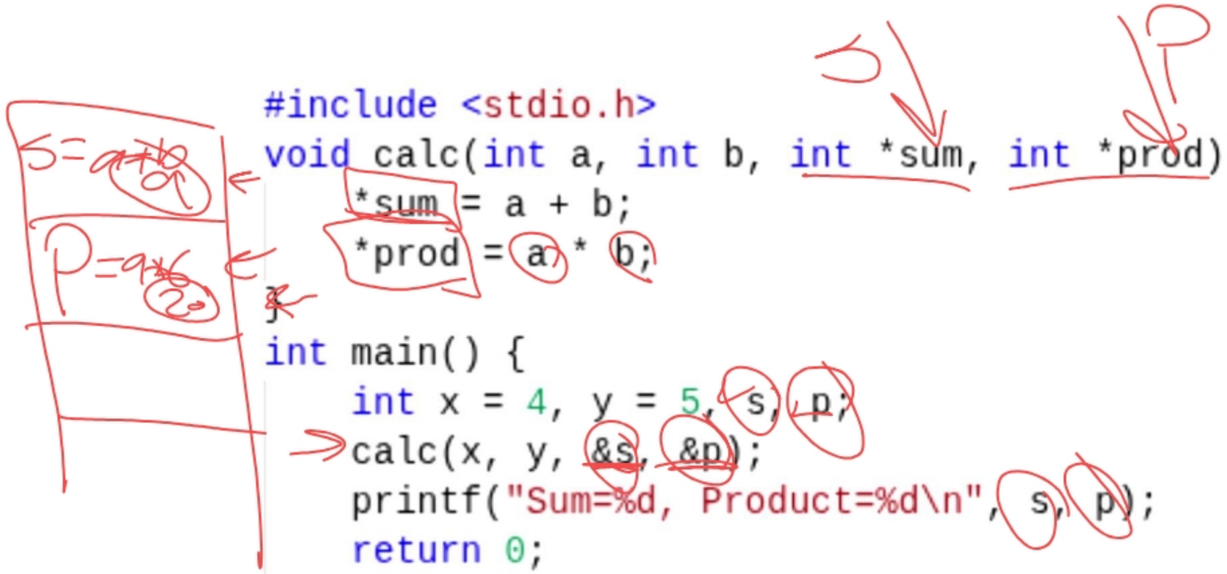
    for (int i = 0; i < 5; i++) {
        printf("%d ", *(p + i));  // access using pointer arithmetic
    }
    return 0;
}
```



3. For Passing an Array to Function

```c
#include <stdio.h>
void printArray(int *arr, int size) {
    for (int i = 0; i < size; i++)
        printf("%d ", arr[i]);
}
int main() {
    int nums[5] = {1,2,3,4,5};
    printArray(nums, 5);
    return 0;
}
```

4. For Returning Multiple Values from a Function



```c
#include <stdio.h>
void calc(int a, int b, int *sum, int *prod) {
    *sum = a + b;
    *prod = a * b;
}
int main() {
    int x = 4, y = 5, s, p;
    calc(x, y, &s, &p);
    printf("Sum=%d, Product=%d\n", s, p);
    return 0;
}
```

1. `malloc(size)`

- Allocates **uninitialized** block of memory of given size (in bytes).
- Returns a pointer to the first byte.

```c
#include <stdio.h>
#include <stdlib.h>

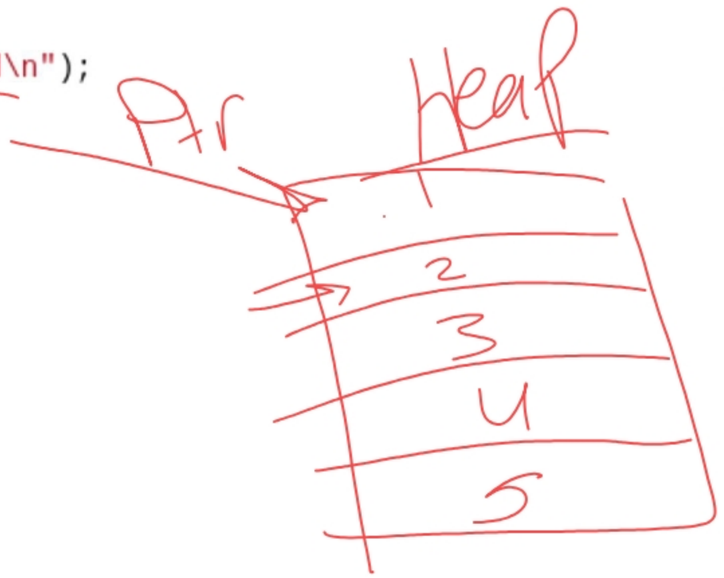
int main() {
    int *p;
    p = (int*) malloc(5 * sizeof(int));  // allocate space for 5 ints

    if (p == NULL) {
        printf("Memory not allocated\n");
        return 1;
    }

    // assign values
    for (int i = 0; i < 5; i++)
        p[i] = i + 1;

    // print values
    for (int i = 0; i < 5; i++)
        printf("%d ", p[i]);

    free(p); // release memory
    return 0;
}
```



- Memory created in **heap**, not stack.
- Without `free(p)`, you'll cause a **memory leak**.

1. `malloc(size)`

- Allocates **uninitialized** block of memory of given size (in bytes).
- Returns a pointer to the first byte.

```c
#include <stdio.h>
#include <stdlib.h>

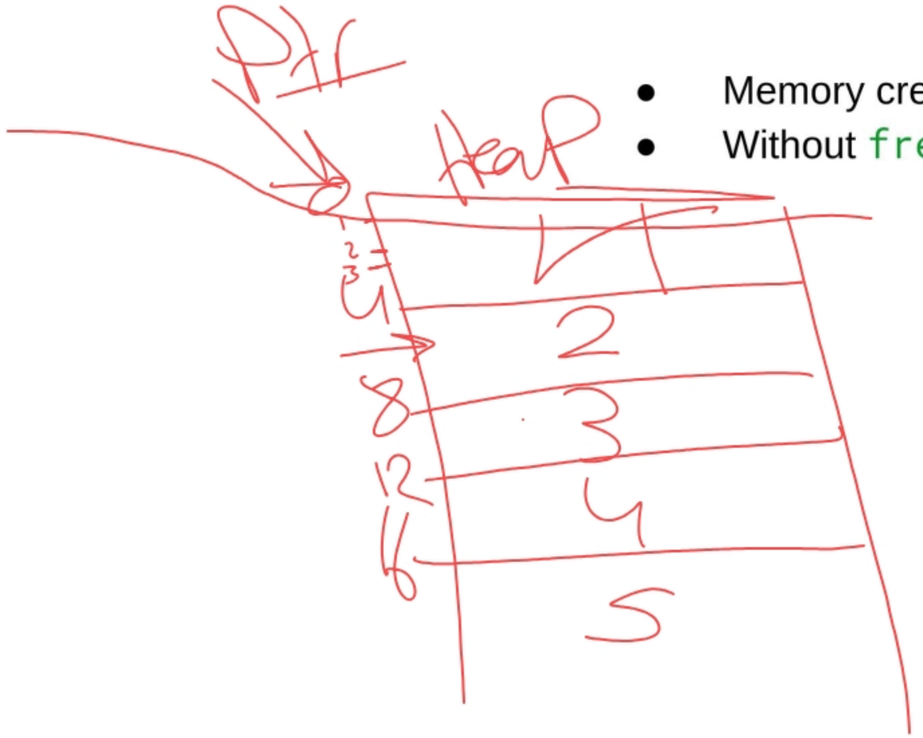
int main() {
    int *p;
    p = (int*) malloc(5 * sizeof(int));  // allocate space for 5 ints

    if (p == NULL) {
        printf("Memory not allocated\n");
        return 1;
    }

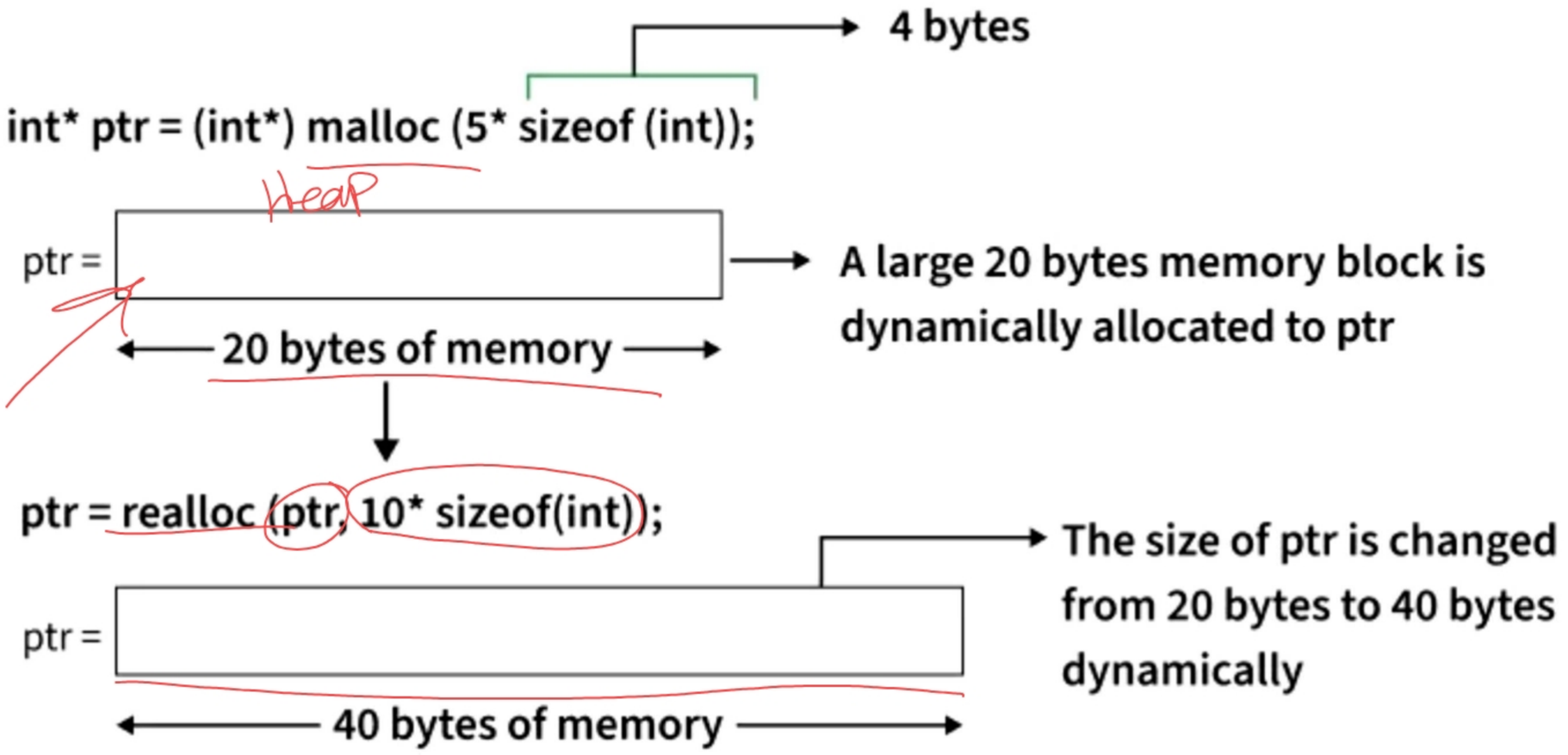
    // assign values
    for (int i = 0; i < 5; i++)
        p[i] = i + 1;

    // print values
    for (int i = 0; i < 5; i++)
        printf("%d ", p[i]);

    free(p); // release memory
    return 0;
}
```



- Memory created in **heap**, not stack.
- Without `free(p)`, you'll cause a **memory leak**.


4 bytes
int* ptr = (int*) malloc (5* sizeof (int));
Heap
ptr =
A large 20 bytes memory block is dynamically allocated to ptr
20 bytes of memory
ptr = realloc (ptr, 10* sizeof(int));
The size of ptr is changed from 20 bytes to 40 bytes dynamically
ptr =
40 bytes of memory

4. `free(ptr)`

- Frees (releases) dynamically allocated memory back to the system.
- Always use `free` after `malloc`/`calloc`/`realloc` to avoid leaks.

```c
#include <stdio.h>
#include <stdlib.h>

int main() {
    int *ptr = (int *)calloc(5, sizeof(int));

    // Do some operations.....
    for (int i = 0; i < 5; i++)
        printf("%d ", ptr[i]);

    // Free the memory after completing
    // operations
    free(ptr);
    printf("%d", *ptr);
    return 0;
}
```



```c
#include <stdio.h>

const int MAX = 4;

int main(){

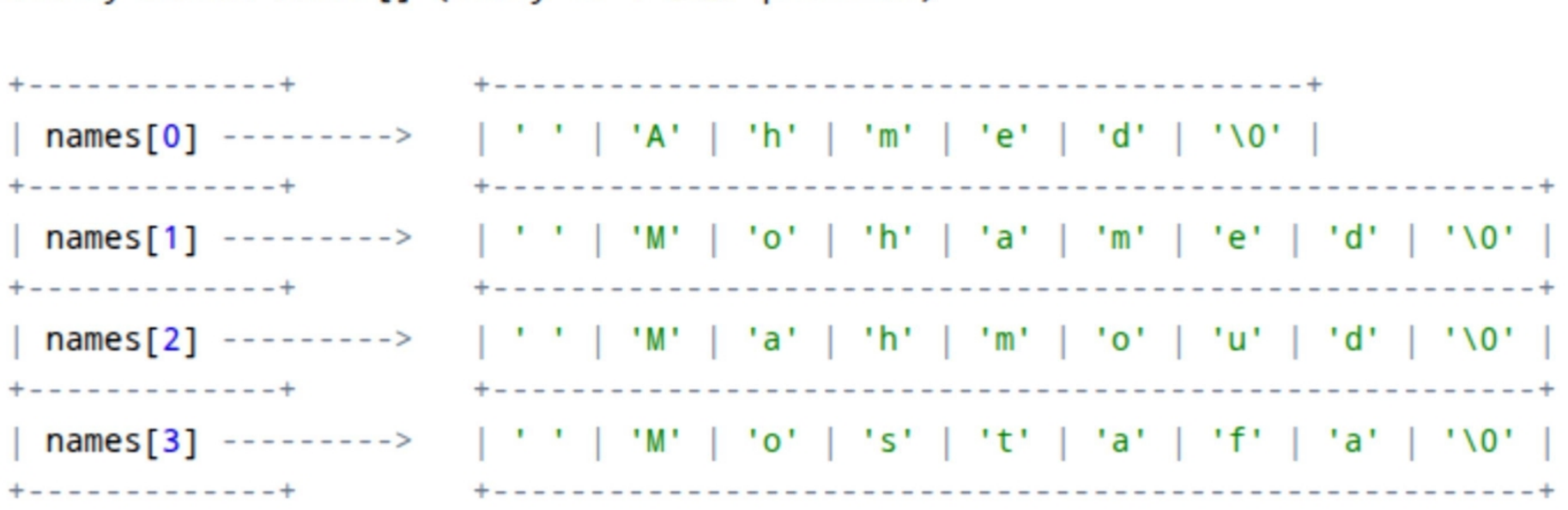
   char *names[] = {
      "Ahmed",
      "Mohamed",
      "Mahmoud",
      "Mostafa"
   };

   int i = 0;

   for(i = 0; i < MAX; i++){
      printf("Value of names[%d] = %s\n", i, names[i]);
   }

   return 0;
}
```

- A pointer to a string **points to the first character** of the string.
- Since strings in C are just arrays of `char` ending with `'\0'`, the pointer allows access to the full string from that first character onward.



```c
#include <stdio.h>

const int MAX = 4;

int main(){

   char *names[] = {
      "Ahmed",
      "Mohamed",
      " Mahmoud",
      " Mostafa"
   };

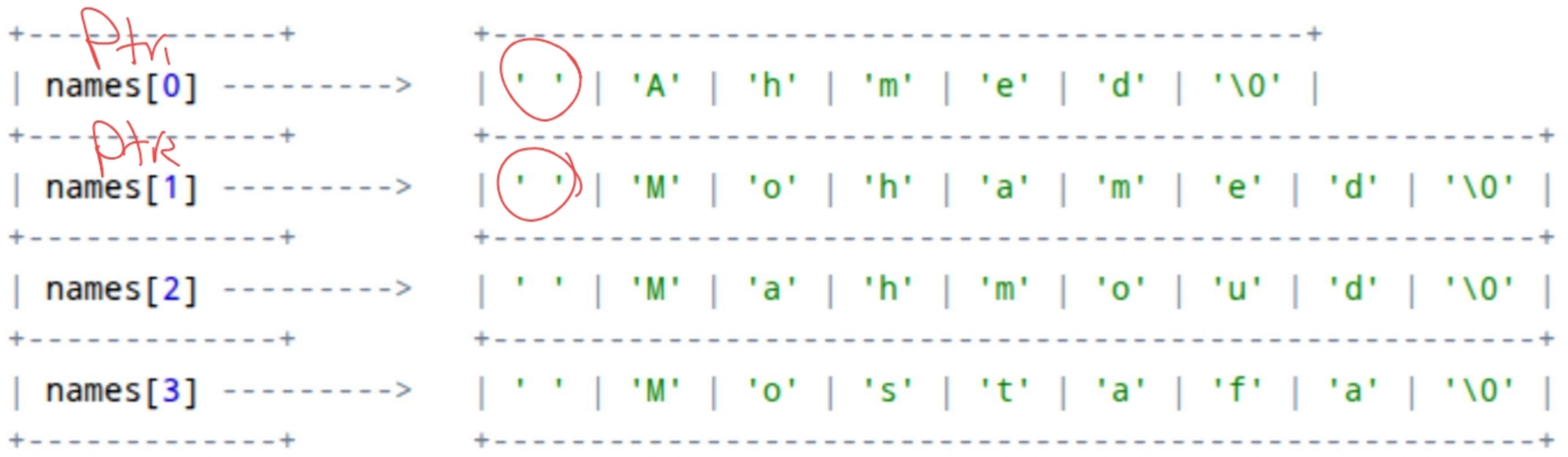
   int i = 0;

   for(i = 0; i < MAX; i++){
      printf("Value of names[%d] = %s\n", i, names[i]);
   }

   return 0;
}
```

- A pointer to a string **points to the first character** of the string.
- Since strings in C are just arrays of `char` ending with `'\0'`, the pointer allows access to the full string from that first character onward.

Memory Block: names[] (array of 4 char pointers)



```c
#include <stdio.h>

const int MAX = 4;

int main(){

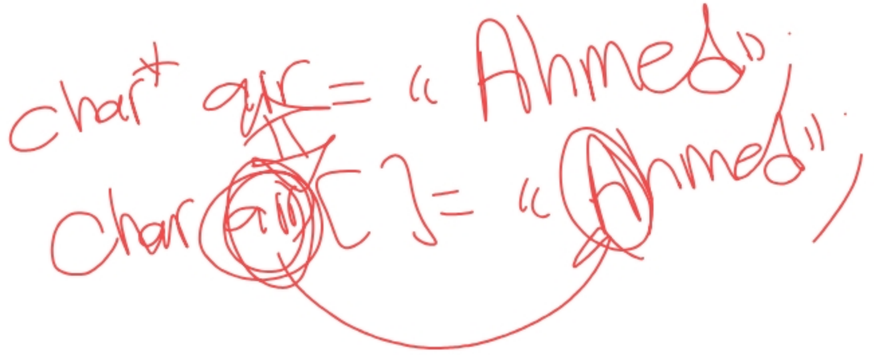
   char *names[] = {
      "Ahmed",
      " Mohamed",
      " Mahmoud",
      " Mostafa"
   };

   int i = 0;

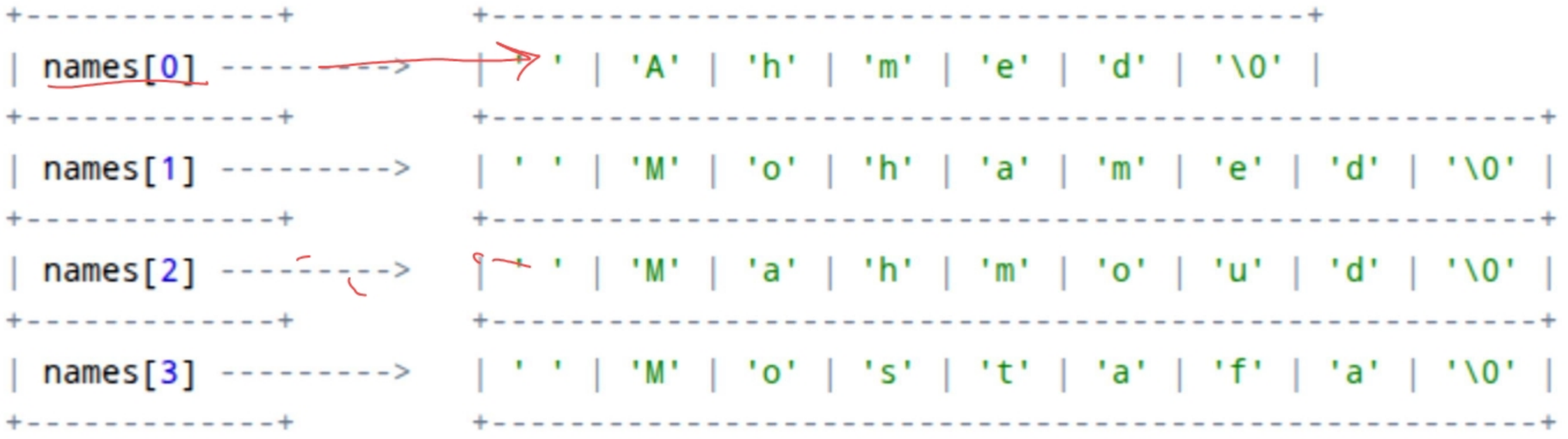
   for(i = 0; i < MAX; i++){
      printf("Value of names[%d] = %s\n", i, names[i]);
   }

   return 0;
}
```

- A pointer to a string **points to the first character** of the string.
- Since strings in C are just arrays of `char` ending with `'\0'`, the pointer allows access to the full string from that first character onward.

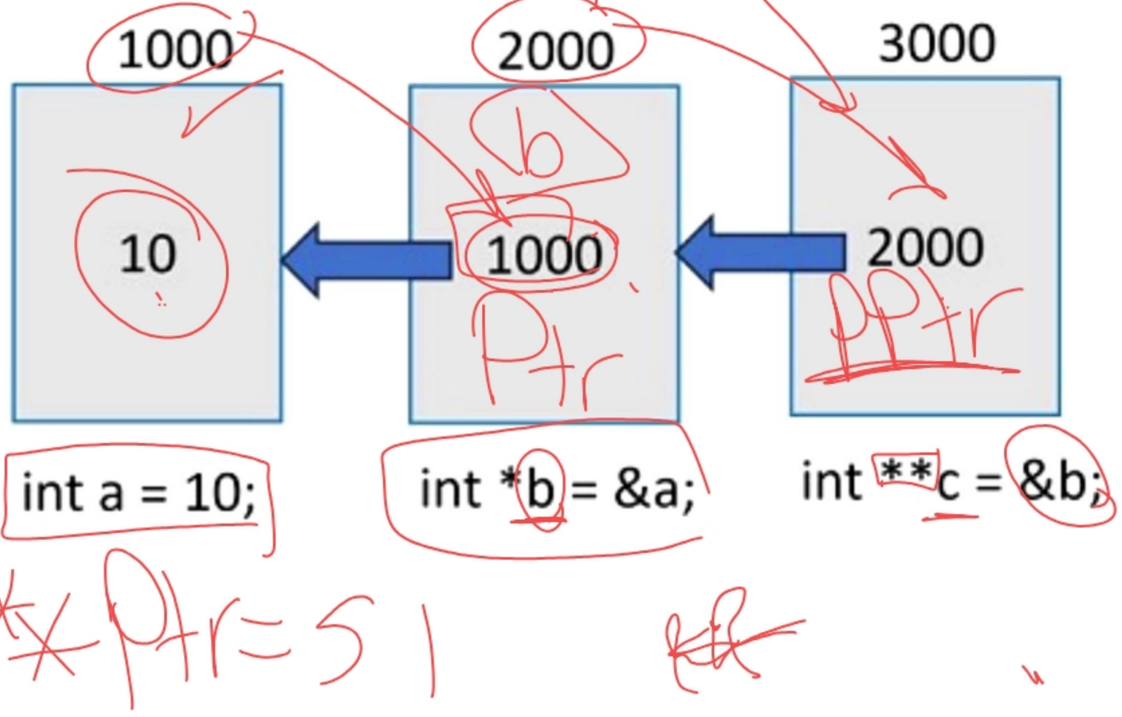


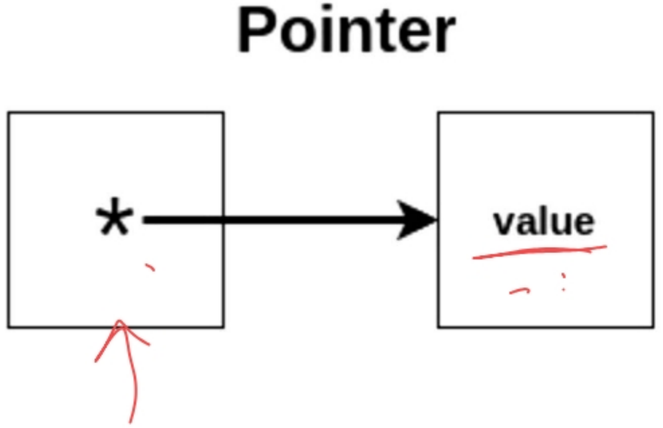
Memory Block: names[] (array of 4 char pointers)

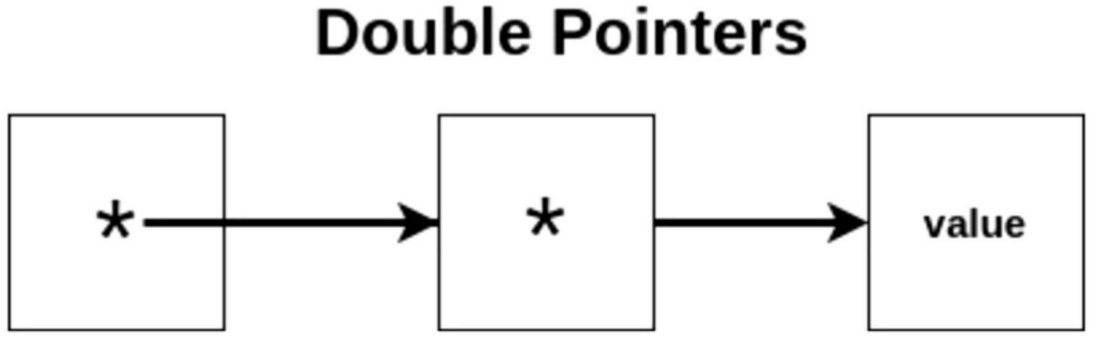


Double Pointer

Pointer to pointer (double pointer) used to store the address of another pointer.

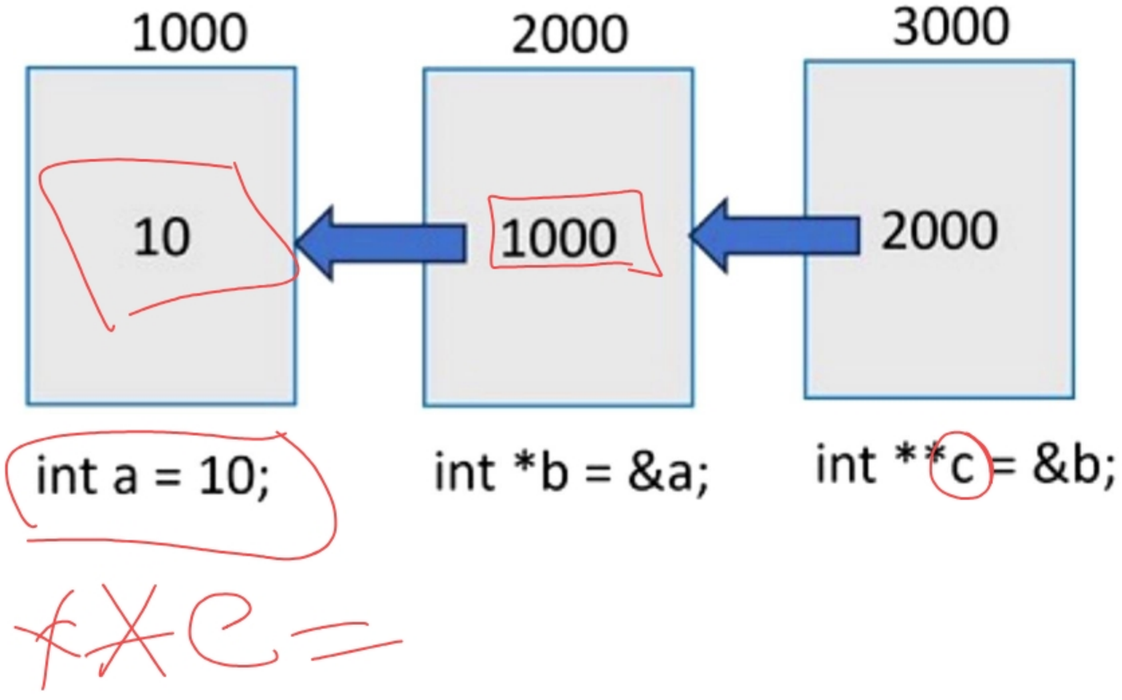


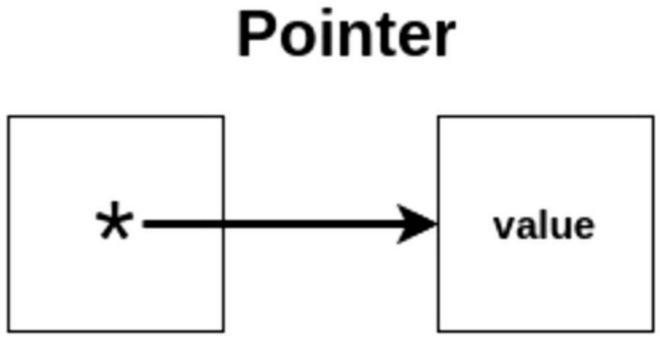


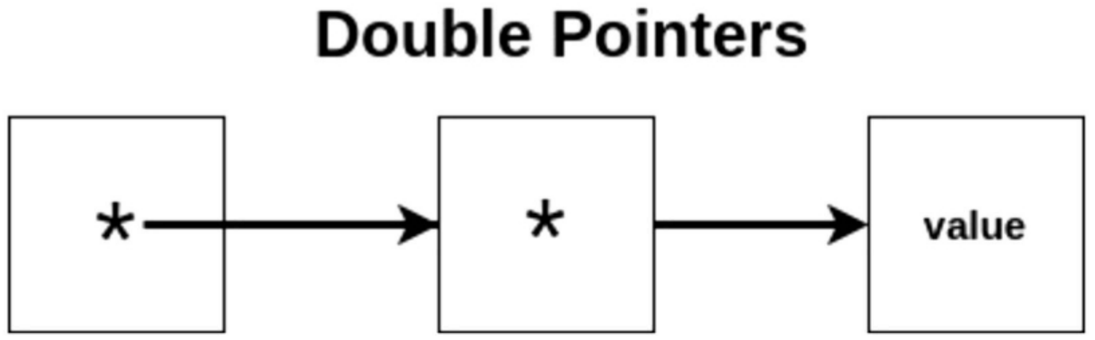


Double Pointer

Pointer to pointer (double pointer) used to store the address of another pointer.







2. Void Pointer (Generic Pointer)

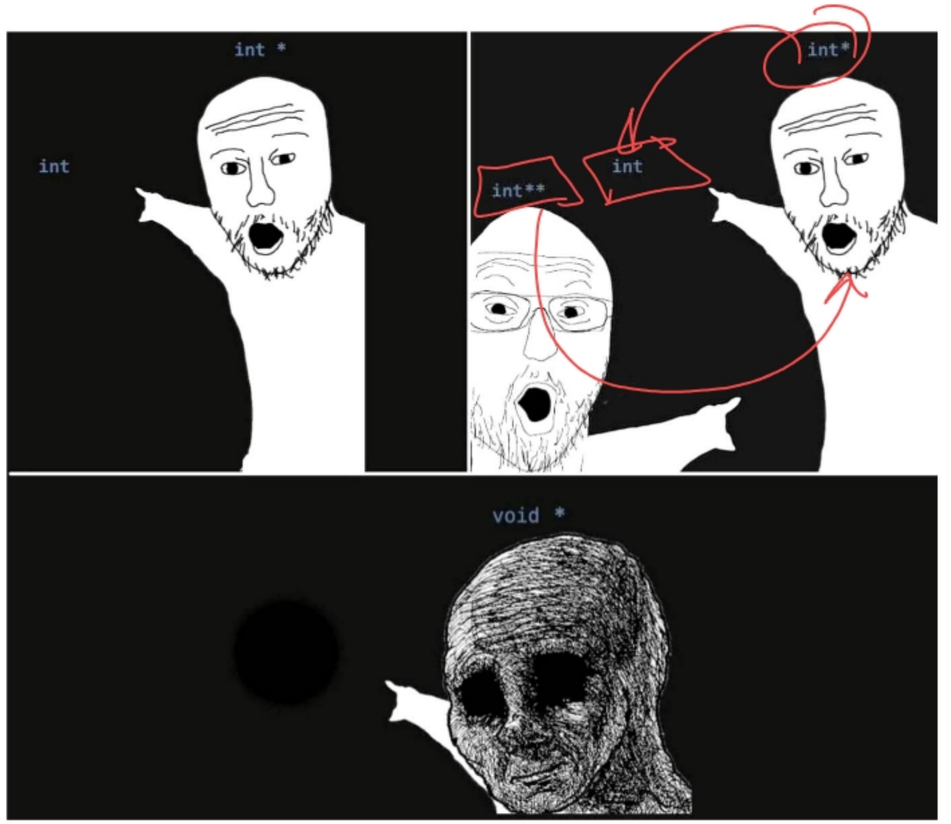
- Special pointer type `void *` that can hold the address of any data type.
- Must be typecast before dereferencing.

```c
#include <stdio.h>
int main() {
    int x = 10;
    float y = 5.5;
    void *ptr;

    ptr = &x;
    printf("Value of x = %d\n", *(int *)ptr);

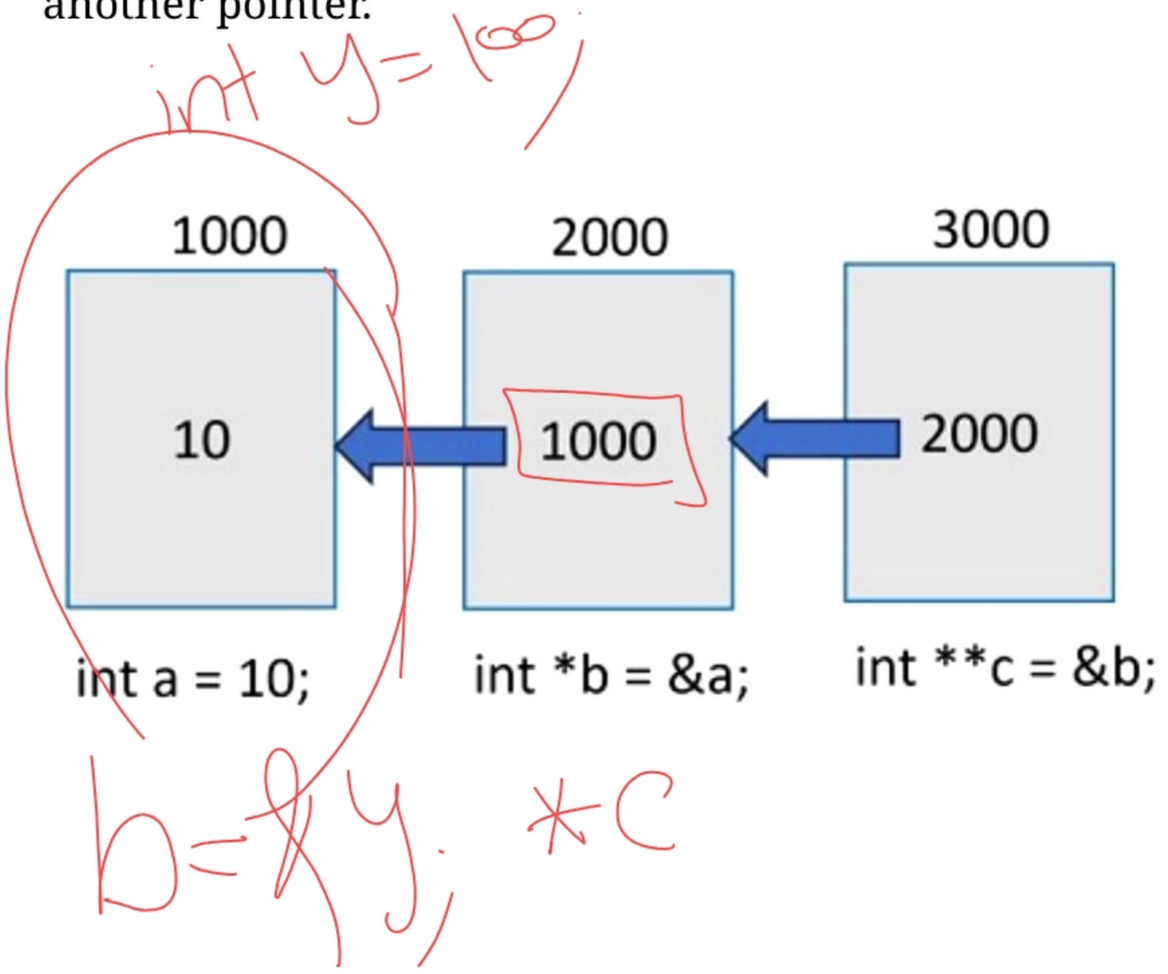
    ptr = &y;
    printf("Value of y = %.1f\n", *(float *)ptr);

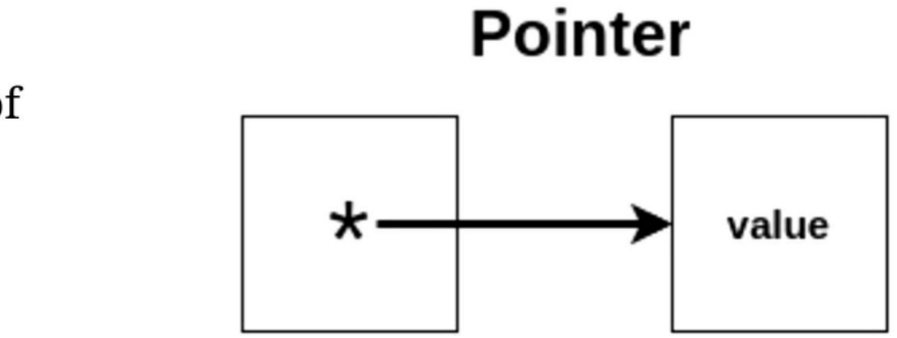
    return 0;
}
```

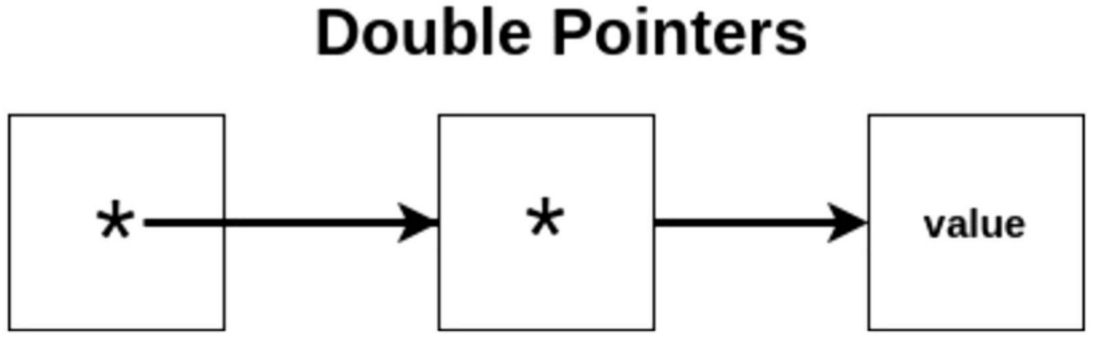


Double Pointer

Pointer to pointer (double pointer) used to store the address of another pointer.

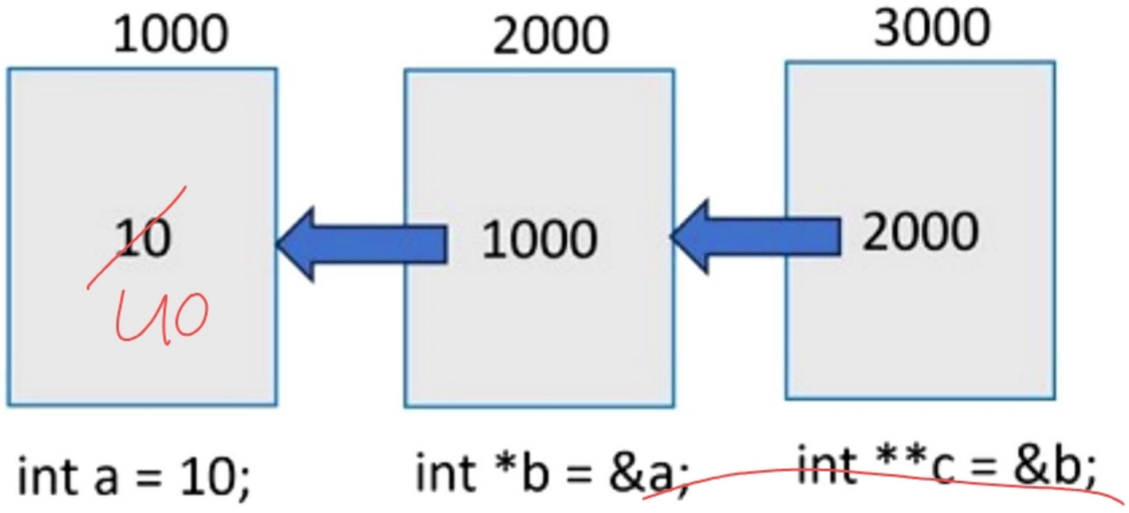




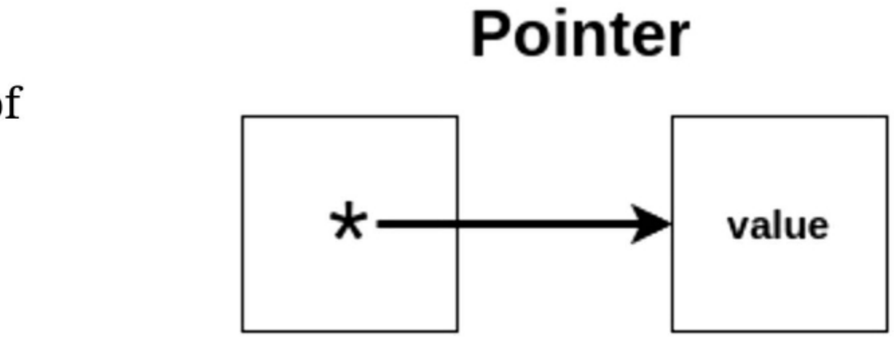


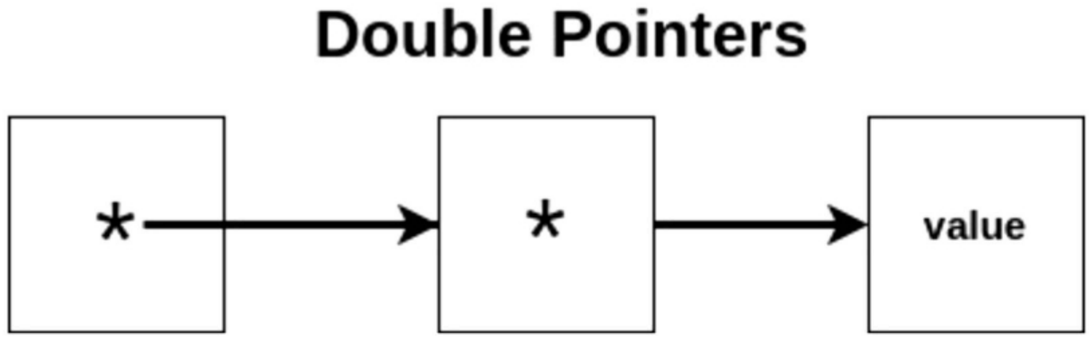
Double Pointer

Pointer to pointer (double pointer) used to store the address of another pointer.









```c
#include <stdio.h>

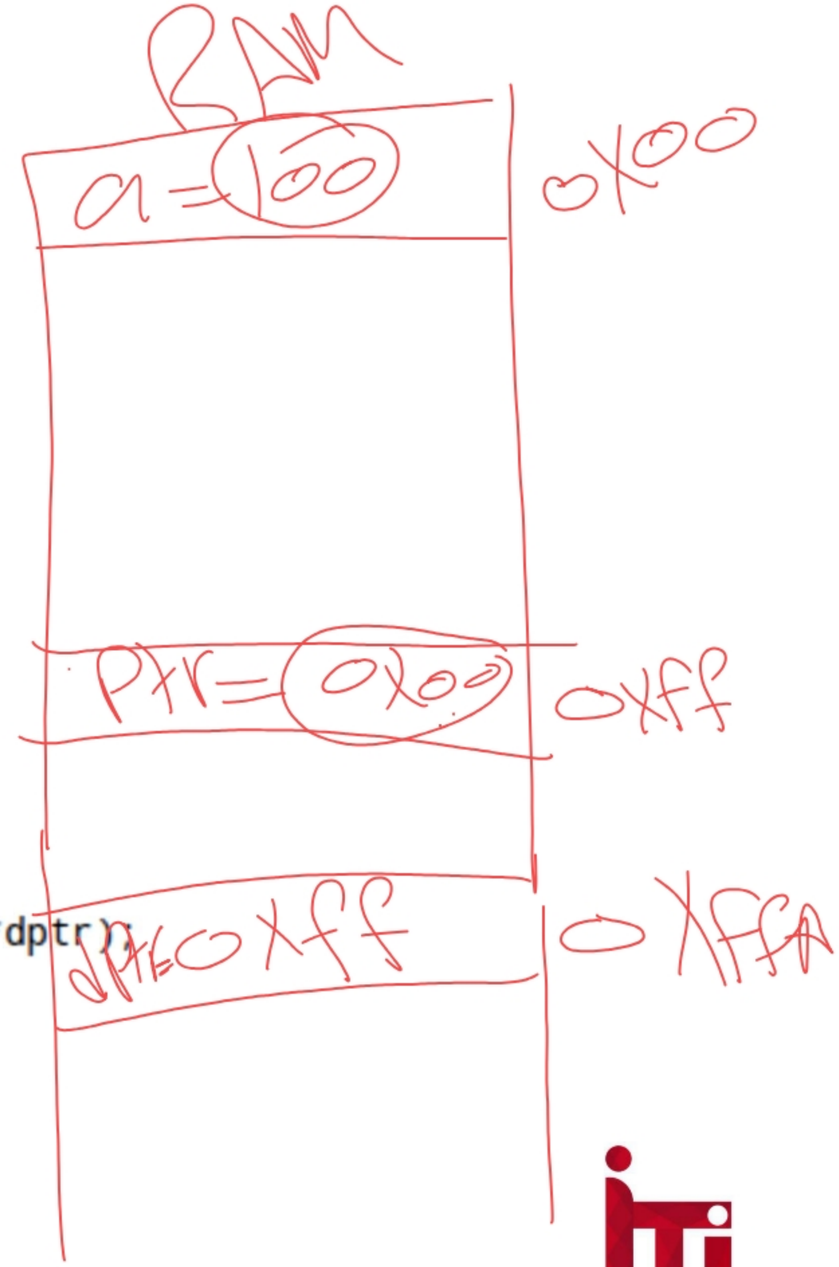
int main() {
    // An integer variable
    int a = 100;

    // Pointer to integer
    int *ptr = &a;

    // Pointer to pointer (double pointer)
    int **dptr = &ptr;

    printf("Value of 'a' is : %d\n", a);
    printf("Value of 'a' using pointer (ptr) is : %d\n", *ptr);
    printf("Value of 'a' using double pointer (dptr) is : %d\n", **dptr);

    return 0;
}
```



```c
#include <stdio.h>

int main() {
    // An integer variable
    int a = 100;

    // Pointer to integer
    int *ptr = &a;

    // Pointer to pointer (double pointer)
    int **dptr = &ptr;

    printf("Value of 'a' is : %d\n", a);
    printf("Value of 'a' using pointer (ptr) is : %d\n", *ptr);
    printf("Value of 'a' using double pointer (dptr) is : %d\n", **dptr);

    return 0;
}
```



Function Pointer (pointer to function)

It is a variable that stores the address of a function. Function pointers can be useful when you want to call a function dynamically. The mechanism of callback functions in C is dependent on the function pointers.

Declaration

Syntax `function_return_type (*Pointer_name) (function_argument_list)`

```c
#include <stdio.h>

// Defining a function
void hello() { printf("Hello World"); }

// The main code
int main() {
  // Declaring a function pointer
  void (*ptr)() = &hello;

  // Calling function using the
  // function poiter
  (*ptr)();

  return 0;
}
```

```c
void hello(){
    printf("Hello World");
}

void (*ptr)() = &hello;
```



```c
int add(int a, int b) {
    return a + b;
}

int (*func_ptr)(int, int);  // Declare the pointer
func_ptr = add;             // Assign function address

// Call the function using the pointer
int result = func_ptr(2, 3); // result = 5
```

```c
typedef int (*operation_func)(int, int);

int add(int a, int b) { return a + b; }

operation_func f = add;
int res = f(5, 6);  // res = 11
```



```c
int add(int a, int b) {
    return a + b;
}

int (*func_ptr)(int, int);  // Declare the pointer
func_ptr = add;             // Assign function address

// Call the function using the pointer
int result = func_ptr(2, 3); // result = 5
```

```c
typedef int (*operation_func)(int, int);

int add(int a, int b) { return a + b; }

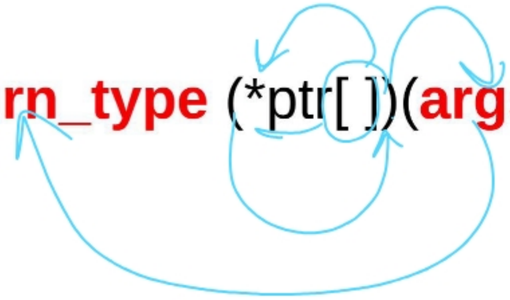
operation_func f = add;
int res = f(5, 6);  // res = 11
```



Array of Function Pointers

An **array of function pointers** lets you store multiple functions (with the same signature) in one place and call them dynamically using an index.

return_type (*ptr[ ])(**args**) = {fun1, fun2, ...};



- `operations`: The name of the array.
- `[4]`: It holds 4 elements.
- Each element is a pointer to a function that takes two `int` arguments and returns an `int`.

```c
#include <stdio.h>

// Define function signatures
int add(int a, int b) { return a + b; }
int subtract(int a, int b) { return a - b; }
int multiply(int a, int b) { return a * b; }
int divide(int a, int b) { return b != 0 ? a / b : 0; }

// Declare array of function pointers
int (*operations[4])(int, int);

int main() {
    // Assign functions to the array
    operations[0] = add;
    operations[1] = subtract;
    operations[2] = multiply;
    operations[3] = divide;

    int x = 20, y = 5;

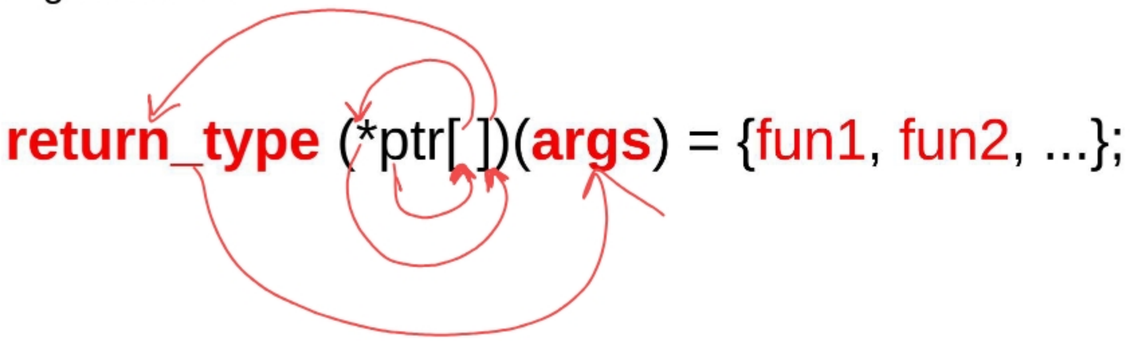
    // Call each operation using the function pointer array
    printf("Add: %d\n", operations[0](x, y));        // add(x, y)
    printf("Subtract: %d\n", operations[1](x, y));   // subtract(x, y)
    printf("Multiply: %d\n", operations[2](x, y));   // multiply(x, y)
    printf("Divide: %d\n", operations[3](x, y));     // divide(x, y)

    return 0;
}
```



Array of Function Pointers

An **array of function pointers** lets you store multiple functions (with the same signature) in one place and call them dynamically using an index.



- `operations`: The name of the array.
- `[4]`: It holds 4 elements.
- Each element is a pointer to a function that takes two `int` arguments and returns an `int`.

```c
#include <stdio.h>

// Define function signatures
int add(int a, int b) { return a + b; }
int subtract(int a, int b) { return a - b; }
int multiply(int a, int b) { return a * b; }
int divide(int a, int b) { return b != 0 ? a / b : 0; }

// Declare array of function pointers
int (*operations[4])(int, int);

int main() {
    // Assign functions to the array
    operations[0] = add;
    operations[1] = subtract;
    operations[2] = multiply;
    operations[3] = divide;

    int x = 20, y = 5;

    // Call each operation using the function pointer array
    printf("Add: %d\n", operations[0](x, y));        // add(x, y)
    printf("Subtract: %d\n", operations[1](x, y));   // subtract(x, y)
    printf("Multiply: %d\n", operations[2](x, y));   // multiply(x, y)
    printf("Divide: %d\n", operations[3](x, y));     // divide(x, y)

    return 0;
}
```

Array of Function Pointers

An **array of function pointers** lets you store multiple functions (with the same signature) in one place and call them dynamically using an index.

return_type (*ptr[ ])(**args**) = {fun1, fun2, ...};

- `operations`: The name of the array.
- `[4]`: It holds 4 elements.
- Each element is a pointer to a function that takes two `int` arguments and returns an `int`.



```c
#include <stdio.h>

// Define function signatures
int add(int a, int b) { return a + b; }
int subtract(int a, int b) { return a - b; }
int multiply(int a, int b) { return a * b; }
int divide(int a, int b) { return b != 0 ? a / b : 0; }

// Declare array of function pointers
int (*operations[4])(int, int);

int main() {
    // Assign functions to the array
    operations[0] = add;
    operations[1] = subtract;
    operations[2] = multiply;
    operations[3] = divide;

    int x = 20, y = 5;

    // Call each operation using the function pointer array
    printf("Add: %d\n", operations[0](x, y));        // add(x, y)
    printf("Subtract: %d\n", operations[1](x, y));   // subtract(x, y)
    printf("Multiply: %d\n", operations[2](x, y));   // multiply(x, y)
    printf("Divide: %d\n", operations[3](x, y));     // divide(x, y)

    return 0;
}
```



Character Pointers

Declaring a Character Pointer

```
char *ptr;
char ch = 'A';
char *ptr = &ch;  // points to the variable ch
char name[] = "Ahmed";
char *ptr = name;
// OR
char *ptr = &name[0];  // same thing
```

char *ptr = "Hello";

- "Hello" is stored in read-only memory.
- You must not modify the contents via ptr.

C Functions
C Function Parameters
C Scope
C Function Declaration
C Functions Challenge
C Math Functions
C Inline Functions
C Recursion
C Function Pointers

C FILES
C Create Files
C Write To Files
C Read Files

C STRUCTURES
C Structures
C Structs Challenge
C Nested Structures
C Structs & Pointers
C Unions
C typedef
C Struct Padding

C ENUMS
C Enums

C MEMORY
C Memory Management

C ERRORS
C Errors
C Error Challenge
C Debugging
C NULL
C Error Handling
C Input Validation

C MORE
C Date
C Random Numbers
C Macros
C Organize Code

A list of all string functions can be found in the table below:

	Function	Description
1	memchr()	Returns a pointer to the first occurrence of a value in a block of memory
2	memcmp()	Compares two blocks of memory to determine which one represents a larger numeric value
3	memcpy()	Copies data from one block of memory to another
4	memmove()	Copies data from one block of memory to another accounting for the possibility that the blocks of memory overlap
5	memset()	Sets all of the bytes in a block of memory to the same value
6	strcat()	Appends one string to the end of another
7	strchr()	Returns a pointer to the first occurrence of a character in a string
8	strcmp()	Compares the ASCII values of characters in two strings to determine which string has a higher value
9	strcoll()	Compares the locale-based values of characters in two strings to determine which string has a higher value
10	strcpy()	Copies the characters of a string into the memory of another string
11	strcspn()	Returns the length of a string up to the first occurrence of one of the specified characters
12	strerror()	Returns a string describing the meaning of an error code
13	strlen()	Return the length of a string
14	strncat()	Appends a number of characters from a string to the end of another string
15	strncmp()	Compares the ASCII values of a specified number of characters in two strings to determine which string has a higher value
16	strncpy()	Copies a number of characters from one string into the memory of another string
17	strpbrk()	Returns a pointer to the first position in a string which contains one of the specified characters
18	strrchr()	Returns a pointer to the last occurrence of a character in a string
19	strspn()	Returns the length of a string up to the first character which is not one of the specified characters
20	strstr()	Returns a pointer to the first occurrence of a string in another string
21	strtok()	Splits a string into pieces using delimiters
22	strxfrm()	Convert characters in a string from ASCII encoding to the encoding of the current locale

REMOVE ADS



⇒ Red function is a must

⇒ blue function is a plus.

Lab 4

- Implement C library string.h

